CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Faculty of Technology & Engineering

Devang Patel Institute of Advance Technology and Research (DEPSTAR)

Data Engineering (ITUC301)

Practical File – ODD Semester 2026–27

Student ID: D25DIT078

Student Name: Jay Kansara

Problem Definition: Study, dissect, and map the complete end-to-end data engineering lifecycle for an enterprise e-commerce platform. Students must trace data states across all five core lifecycle stages: data generation, ingestion, storage, transformation, and serving. The solution requires creating a detailed architectural data flow diagram along with a system manifesto documenting cross-cutting lifecycle undercurrents (security, observability, and data privacy). This initial structure must be committed to a personal Git repository.

Dataset/Test Data: Mock enterprise e-commerce dataset schema definition (containing transactional database entities, clickstream tracking JSON payloads, and third-party inventory log streams).

Tools/Technology to Be Used: Draw.io, Lucid chart, Git, and GitHub/GitLab.

SYSTEM OVERVIEW

Mosaic is an AI-powered collaborative event and trip planning platform designed to simplify group planning by bringing all essential features into a single application. Users can create events or trips, invite participants, and provide details such as destination, budget, duration, and preferences. Based on these inputs, the AI generates personalized itineraries and recommendations.

The platform enables real-time collaboration through shared planning, group voting, and live updates. It also includes budget tracking and automatic expense splitting to help participants manage costs efficiently. After the event, users can upload photos and videos, which the AI organizes into a chronological timeline with generated summaries, creating a digital memory of the experience.

Built using React, Node.js, Express.js, and Supabase, Mosaic provides a centralized, intelligent, and user-friendly solution for planning, managing, and preserving group trips and events.

Level 0 DFD – Context Diagram

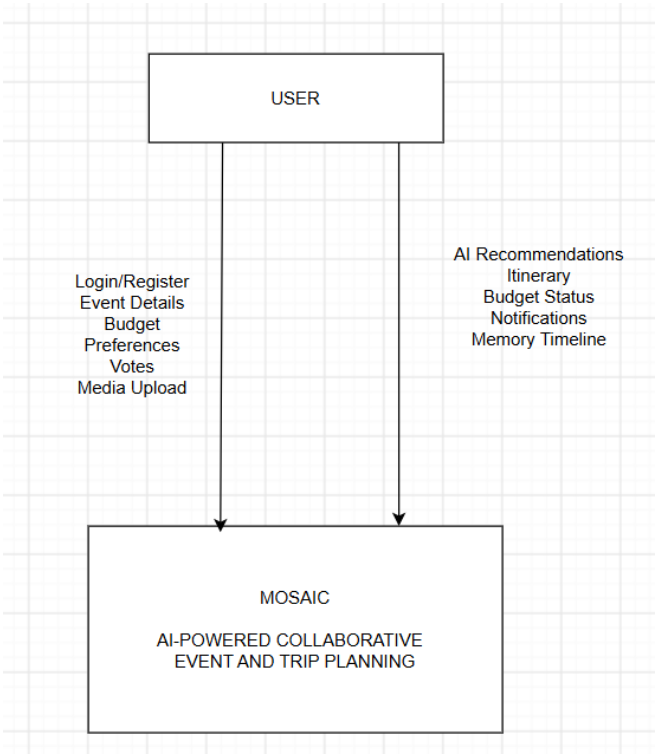


Figure 1.1 Context Diagram : Mosaic

Level 1 DFD – Major Process Decomposition

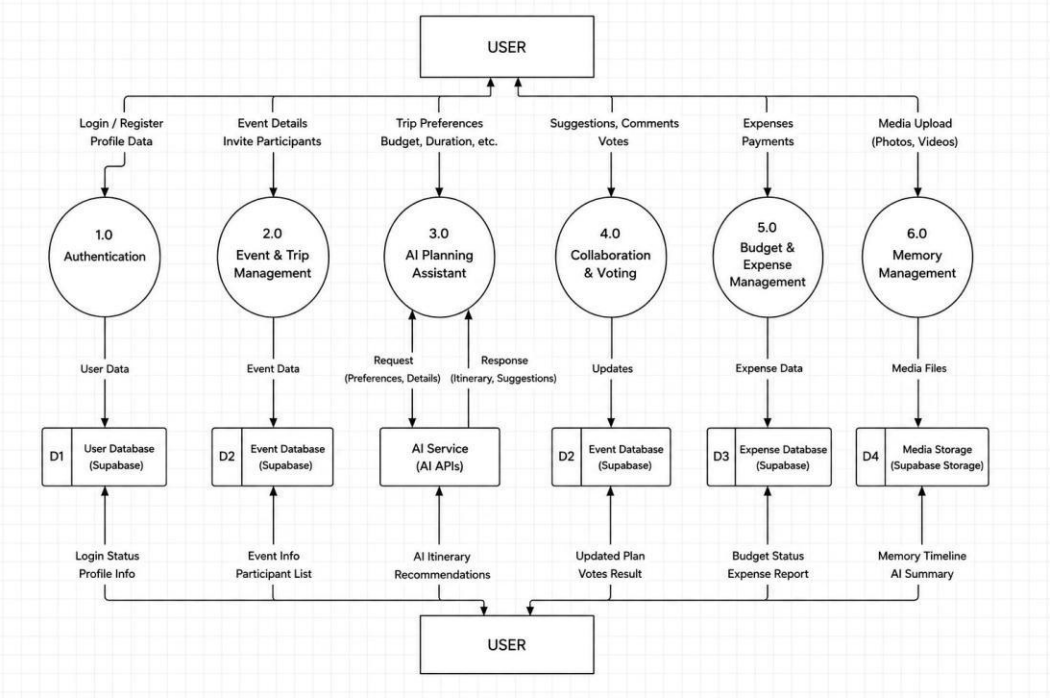


Figure 1.2 Level 1 DFD : Mosaic

Level 2 DFD Expand 2.0 Event & Trip Management

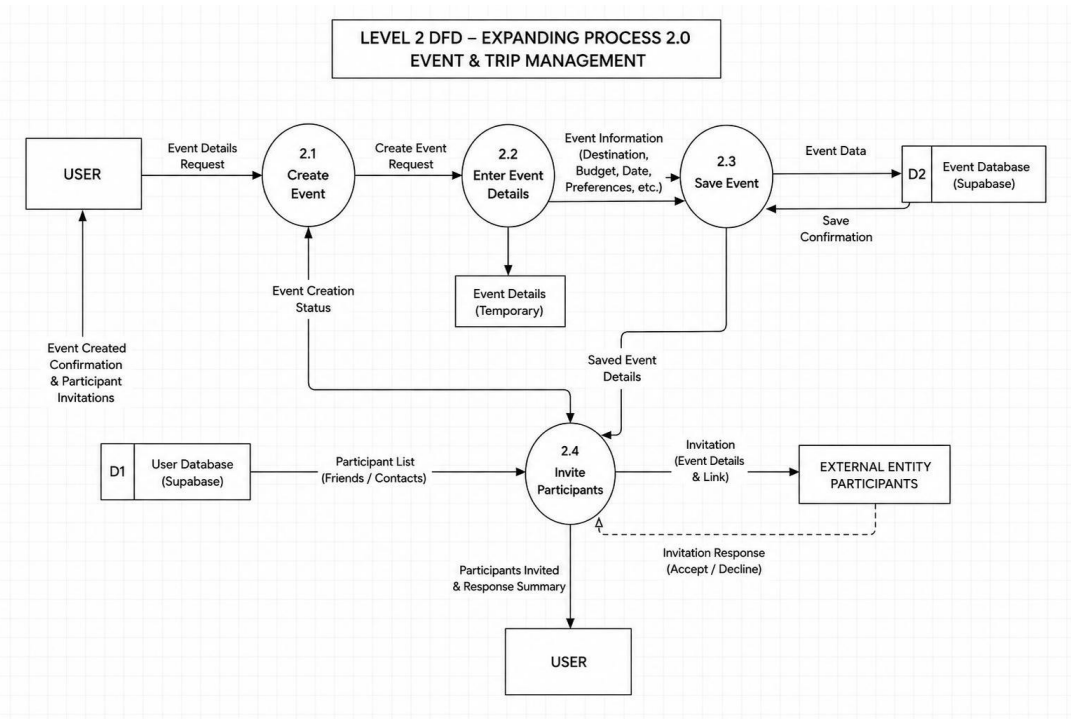


Figure 1.3 Level 2 DFD : Mosaic

Q1. How do the technical demands of downstream serving layers directly influence ingestion and storage design decisions during the early generation phases?

Answer:
The downstream serving layer determines what data needs to be collected and how it should be stored. In Mosaic, since users need real-time AI recommendations, collaboration, and budget tracking, the system collects structured data such as event details, user preferences, expenses, and votes. This data is stored in Supabase (PostgreSQL) for fast retrieval and efficient processing by the backend and AI modules, ensuring quick responses and a smooth user experience.

Q2. Differentiate between the broader data lifecycle and the technical data engineering lifecycle within an enterprise.

Answer:

Data Lifecycle:
Focuses on how data is managed from creation to deletion.
Stages:

- Data Collection
- Storage
- Processing
- Sharing
- Usage
- Archiving
- Deletion

Data Engineering Lifecycle:
Focuses on the technical processes that prepare data for applications and analytics.
Stages:

- Data Ingestion
- Data Storage
- Data Transformation
- Data Processing
- Data Serving
- Monitoring & Maintenance

Difference:
The data lifecycle describes the overall journey of data, while the data engineering lifecycle focuses on the technical infrastructure and workflows used to manage and deliver that data.

Q3. Identify how the undercurrents of security and data observability should be enforced at each lifecycle boundary.

Answer:

Security:

- Secure user authentication using Supabase Authentication.
- Role-based access so only authorized users can access or modify event data.
- Encrypt data during transmission (HTTPS) and protect stored data.
- Validate user inputs to prevent attacks.

Data Observability:

- Monitor API requests and system logs.
- Track database performance and errors.
- Detect failed AI requests or processing issues.
- Monitor application performance to ensure reliable collaboration and recommendations.

In Mosaic, security protects user information and event data, while observability helps detect problems early and keeps the platform reliable and responsive.